

SISTEMA DE MONITOREO CON MÁQUINA DE ESTADOS FINITOS IMPLEMENTADO EN ARDUINO MEGA 2560



Universidad
del Cauca

Elaborado por:

Andrea Fernanda Gómez
Juan Sebastián Muñoz Ruiz
Daniel alexander Chaguendo

Universidad del Cauca
Facultad de Ingeniería Electrónica y Telecomunicaciones
Ingeniería de sistemas
Popayán, Cauca, Colombia

1. Introducción

El presente informe documenta el diseño, implementación y validación de un sistema de monitoreo basado en una Máquina de Estados Finitos (MEF) sobre una plataforma Arduino Mega 2560. El proyecto fue desarrollado en el marco del curso de Arquitectura Computacional del programa de Ingeniería de Sistemas de la Universidad del Cauca, período 2026-1.

El sistema integra múltiples sensores —temperatura, humedad, luz ambiental, presencia magnética y sonido— con mecanismos de seguridad de acceso mediante teclado matricial 4x4 y módulo RFID MFRC522. La arquitectura de software elimina el uso de `delay()` o `millis()` bloqueante mediante la librería `AsyncTaskLib`, garantizando una ejecución concurrente eficiente dentro de las restricciones del microcontrolador.

2. Planteamiento del Problema

El diseño de sistemas electrónicos eficientes y versátiles requiere soluciones capaces de gestionar múltiples variables de manera simultánea. Se buscó implementar una Máquina de Estados Finitos (MEF) sobre Arduino capaz de manejar eventos complejos derivados de la interacción entre variables ambientales y de seguridad.

2.1 Variables monitoreadas

- Temperatura y humedad relativa del ambiente (sensor DHT11)
- Nivel de iluminación ambiental (fotoresistor analógico, pin A2)
- Detección de campos magnéticos / apertura de puertas (sensor Hall, pin 31)
- Detección de sonido / intrusos (sensor de micrófono digital, pin 33)
- Autenticación de usuarios (teclado 4x4 + módulo RFID MFRC522 vía SPI)

2.2 Objetivos del sistema

- Detectar cambios en las condiciones ambientales y de seguridad en tiempo real.
- Implementar transiciones claras y efectivas entre estados.
- Gestionar control de acceso multi-factor (contraseña + RFID + horario).
- Almacenar la configuración (contraseña y horario) de forma persistente en EEPROM.
- Generar alertas visuales y sonoras ante eventos críticos.

3. Arquitectura del Sistema

3.1 Descripción general

El sistema sigue una arquitectura modular de cuatro capas: sensores de entrada, procesamiento central (MEF en Arduino Mega 2560), actuadores de salida e interfaz de usuario. Cada capa se comunica a través de señales digitales y analógicas, gestionadas por tareas periódicas asíncronas que se activan o detienen según el estado activo de la máquina.

3.2 Componentes de hardware

La siguiente tabla describe los componentes del sistema, su función y los pines del Arduino Mega 2560 utilizados:

Componente	Función	Pin(es)	Estado relacionado
Sensor Temperatura	Temperatura y humedad ambiental	Pin 6 (digital)	Monitor Ambiental → Alarma
Sensor Fotorresistor	Nivel de luz ambiental	Pin A2 (analógico)	Monitor Ambiental → Alarma
Sensor Hall	Detección de campo magnético	Pin 31 (digital)	Monitor Intrusos → Alarma
Sensor Micrófono	Detección de sonido	Pin 33 (digital)	Monitor Intrusos → Alarma
Módulo RFID MFRC522	Autenticación por tarjeta	Pines 53, 49 (SPI)	Estado Inicio
Teclado matricial 4×4	Entrada de contraseña y menú	Pines 24–38	Inicio / Config / Ambiental
Pantalla LCD 16×2	Visualización de información	Pines 2–5, 11, 12	Todos los estados
LED RGB	Indicador visual de estado	Pines 9 (R), 46 (G), 44 (B)	Alarma / Bloqueo / Acceso OK
Buzzer	Alerta sonora	Pin 7	Alarma (1000 Hz) / Bloqueo (500 Hz)
Botón físico	Desbloqueo manual del sistema	Pin 40 (INPUT_PULLUP)	Estado Bloqueo
Arduino Mega 2560	Unidad central de procesamiento	—	Control de toda la MEF

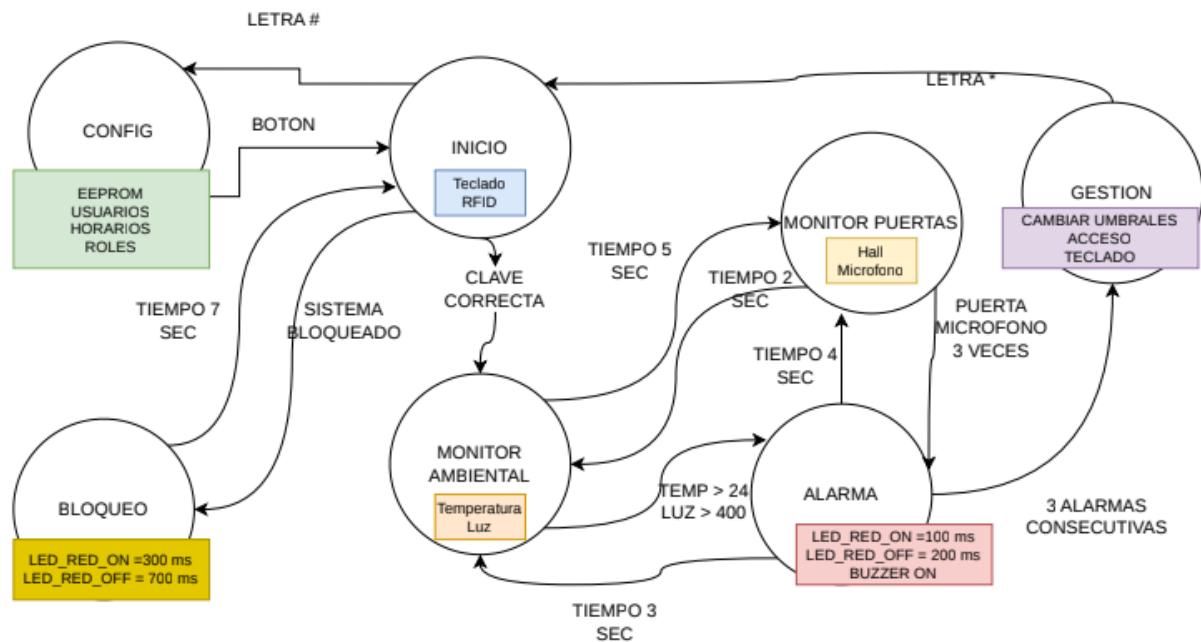
3.3 Estructura del código fuente

El proyecto se organiza en cuatro archivos que el entorno Arduino IDE concatena automáticamente:

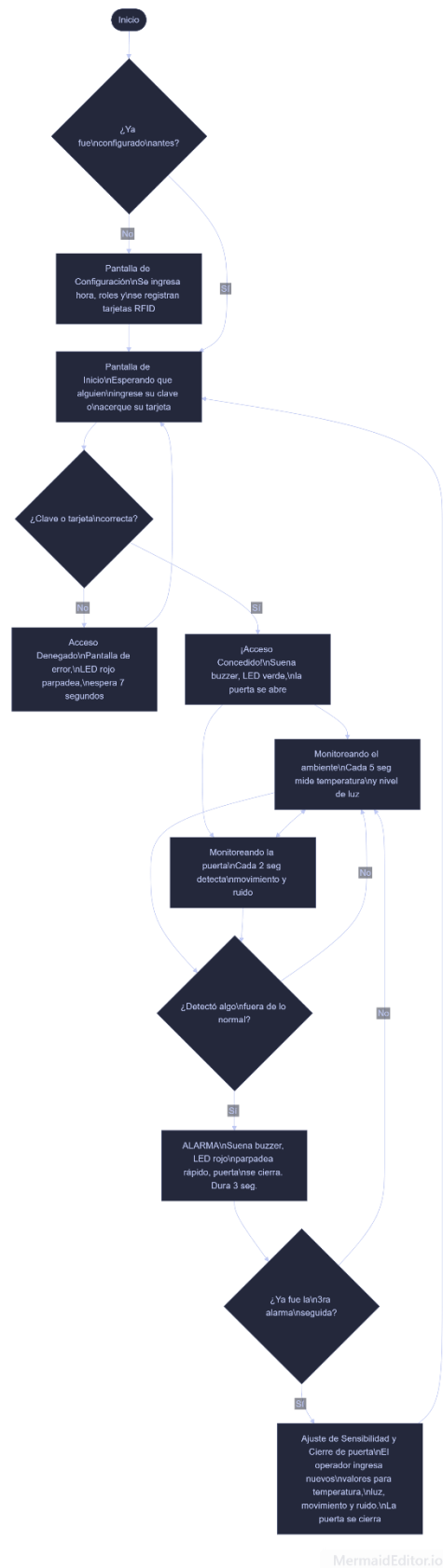
- **SistemaMonitoreo.ino:** Archivo principal. Declara periféricos, instancias de AsyncTask y contiene setup() y loop().
- **statemachine.ino:** Define las transiciones y los callbacks on_enter / on_exit de cada estado.

- **fsm.h:** Declara los enumerados State e Input, y los prototipos de funciones de la MEF.
- **pinout.h:** Centraliza pines, timeouts, umbrales de sensores y direcciones de EEPROM.

4. Máquina de Estados Finitos (MEF)



4.05: Diagrama de Flujo del Sistema



MermaidEditor.io

4.1 Estados del sistema

La MEF implementa 6 estados, cada uno con tareas asíncronas específicas que se activan al entrar y se detienen al salir del estado:

Estado	Descripción	Actividad interna	Salida visual
Inicio	Seguridad: autenticación de usuario	Teclado y RFID activos; máx. 3 intentos	LED apagado
Config	Selección de modo de operación	Teclado activo; opciones * y #	LED verde breve al entrar
Monitor Ambiental	Lectura de sensores ambientales	KY-013 y fotoresistor cada 500 ms	LCD con T° y Luz
Monitor Intrusos	Vigilancia activa de intrusos	Hall y Mic cada 300 ms; timeout 5 s	LCD con estado Hall/Mic
Alarma	Activación de alertas	LED rojo 300/700 ms; buzzer 1000 Hz	LED rojo parpadeante
Bloqueo	Acceso bloqueado por intentos	Espera botón físico; buzzer 500 Hz	LED rojo 100/500 ms

4.2 Señales de entrada

El sistema reacciona a 12 señales de entrada definidas en el enumerado Input de fsm.h. La variable global currentInput almacena la señal activa en cada ciclo del loop():

```

SIG_CLAVE_CORRECTA // Clave + UID RFID + horario válidos
SIG_CONFIG_OK      // Config confirmada → Monitor Ambiental
SIG_TECLA_asterisco // Tecla * presionada
SIG_TECLA_HASH     // Tecla # presionada
SIG_ALARMA_COND    // Temp > 32°C y Luz > 100 ADC
SIG_INTRUSOS       // Hall o Mic activo ≥ 3 veces
SIG_TIMEOUT_AMBIENT // Timeout 4 s en estado Alarma
SIG_TIMEOUT_INTRUS // Timeout 2 s en estado Alarma
SIG_ACTIVACIONES    // 3 alarmas en ventana de 12 s
SIG_TIMEOUT_INTRUS2 // Timeout 5 s en Monitor Intrusos
SIG_BLOQUEADO       // 3 intentos fallidos de clave
SIG_BOTON           // Botón físico presionado

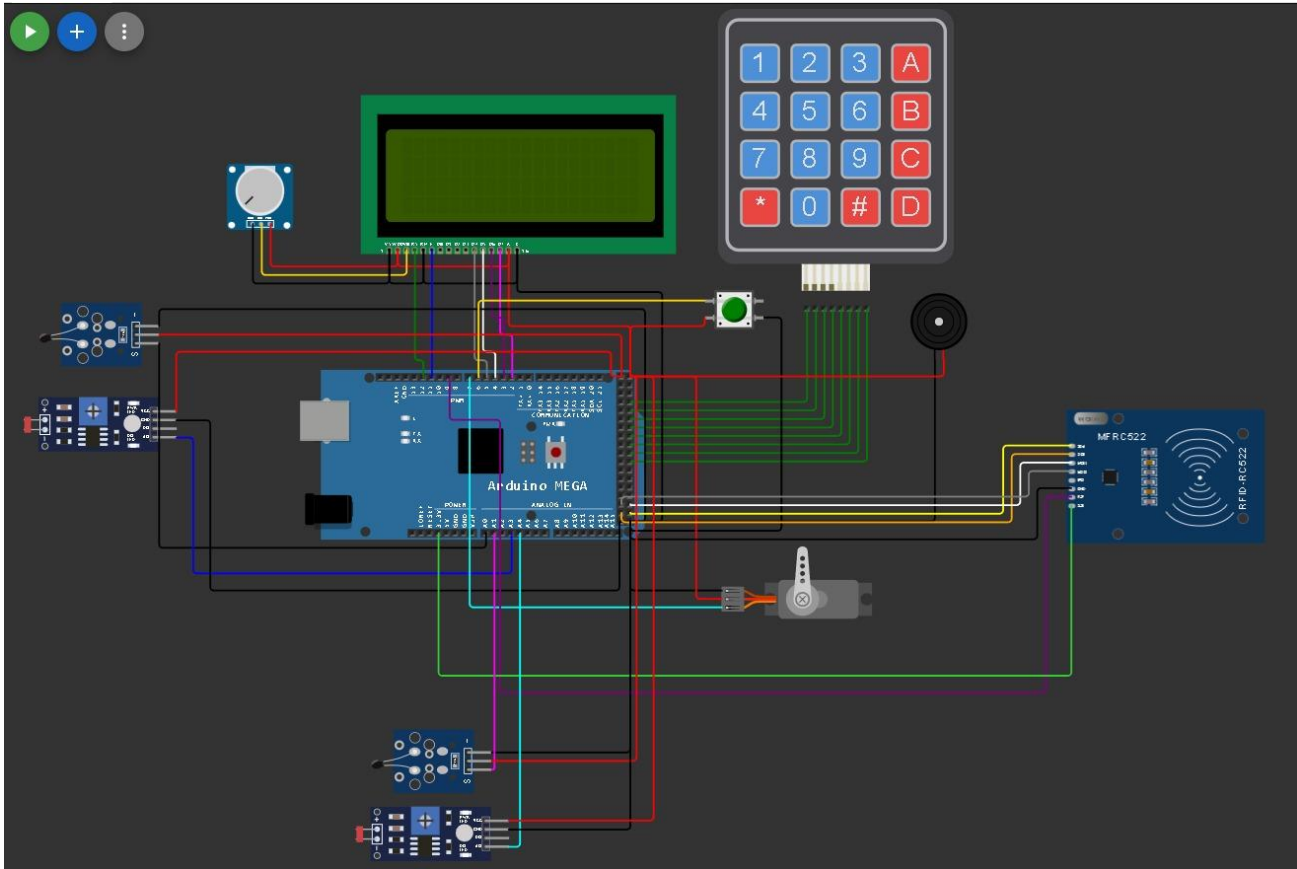
```

4.3 Tabla de transiciones

La MEF define 12 transiciones. La tabla siguiente describe el estado de origen, el estado de destino y la condición que activa cada transición:

Estado Origen	Estado Destino	Condición de Transición
Inicio	Config	Clave 4 dígitos correcta + UID RFID válido + horario válido
Inicio	Bloqueo	3 intentos fallidos de contraseña (MAX_INTENTOS_CLAVE = 3)
Bloqueo	Inicio	Botón físico presionado (Pin 40, INPUT_PULLUP → LOW)
Config	Monitor Ambiental	Tecla * presionada (horario dentro del rango EEPROM)
Config	Monitor Intrusos	Tecla # presionada
Monitor Ambiental	Config	Tecla * presionada por el operador
Monitor Ambiental	Alarma	Temp > 32 °C Y Luz > 100 (ADC) simultáneamente
Monitor Intrusos	Monitor Ambiental	Timeout 5 segundos sin detección de intrusos
Monitor Intrusos	Alarma	Sensor Hall o Mic activo ≥ 3 veces consecutivas
Alarma	Monitor Ambiental	Timeout 4 s (cuando alarma vino de Monitor Ambiental)
Alarma	Monitor Intrusos	Timeout 2 s (cuando alarma vino de Monitor Intrusos)
Alarma	Inicio	3 activaciones de Alarma dentro de una ventana de 12 segundos

4.5 Diagrama de conexiones



4.4 Descripción de los estados

Estado: Inicio

Punto de partida del sistema. Activa las tareas de lectura del teclado (cada 100 ms) y del módulo RFID (cada 200 ms). El usuario debe ingresar una contraseña de 4 dígitos (confirmada con #) y acercar una tarjeta RFID registrada. Si ambas autenticaciones son correctas y la hora actual está en el rango horario almacenado en EEPROM, el sistema emite SIG_CLAVE_CORRECTA y transiciona a Config. Tres intentos fallidos consecutivos generan SIG_BLOQUEADO.

Estado: Config

Permite al operador elegir el modo de operación: tecla * lleva al Monitor Ambiental (previa verificación de horario) y tecla # al Monitor de Intrusos directamente. En versiones futuras, este estado puede extenderse para modificar la contraseña, el UID autorizado y el rango horario desde el teclado.

Estado: Monitor Ambiental

Lee la temperatura y la luz cada 500 ms mostrando los valores en la LCD. Si temperatura < 20 °C y luz < 100 ADC simultáneamente, se genera SIG_ALARMAS_COND con alarmaVieneDeIntrusos = false. La tecla * regresa al estado Config.

Estado: Monitor Intrusos

Lee los sensores Hall y de micrófono cada 300 ms. Si cualquiera se activa en 3 ocasiones consecutivas, genera SIG_INTRUSOS con alarmaVieneDeIntrusos = true. Tras 5 segundos sin detección, el timeout genera SIG_TIMEOUT_INTRUS2 y el sistema regresa al Monitor Ambiental.

Estado: Alarma

Activa el LED rojo con parpadeo asimétrico de 300 ms ON / 700 ms OFF mediante dos tareas coordinadas (taskParpadeoAlarma y taskAlarmaLedOFF), y el buzzer a 1000 Hz continuamente. Registra cada activación; si se acumulan 3 en 12 segundos se emite SIG_ACTIVACIONES y el sistema regresa a Inicio. De lo contrario, el timeout determina el retorno: 4 s si la alarma fue ambiental, 2 s si fue por intrusión.

Estado: Bloqueo

Activa el LED rojo con parpadeo 100 ms ON / 500 ms OFF y el buzzer a 500 Hz. El sistema permanece bloqueado hasta que se detecta una pulsación del botón físico (pin 40 a LOW), lo que genera SIG_BOTON y regresa al estado Inicio. Este mecanismo previene ataques de fuerza bruta.

5. Implementación de Software

5.1 Librerías utilizadas

Librería	Propósito	Funciones clave
Keypad	Lectura del teclado matricial 4x4	getKey(), makeKeymap()
LiquidCrystal	Control de pantalla LCD 16x2 (modo 4 bits)	begin(), print(), clear(), setCursor()
DHT	Lectura de temperatura y humedad	begin(), readTemperature(), readHumidity()
MFRC522	Comunicación con módulo RFID via SPI	PCD_Init(), PICC_IsNewCardPresent(), ReadCardSerial()
StateMachineLib	Gestión de la máquina de estados finitos	AddTransition(), SetOnEntering(), SetOnLeaving(), Update()
AsyncTaskLib	Tareas asíncronas sin delay() bloqueante	Start(), Stop(), Update(), reset()
EEPROM	Almacenamiento persistente en memoria interna	read(), write()

Librería	Propósito	Funciones clave
SPI	Protocolo de comunicación hardware para RFID	begin() (bus SPI hardware del Mega)

5.2 Tareas asíncronas (AsyncTaskLib)

Todas las acciones periódicas se implementan como instancias de AsyncTask, eliminando delay(). En el loop(), todas las tareas llaman a .Update() para comprobar si ha llegado su turno de ejecución:

```
AsyncTask taskLeerTeclado (100, true, task_LeerTeclado);
AsyncTask taskLeerRFID (200, true, task_LeerRFID);
AsyncTask taskLeerAmbiental (500, true, task_LeerAmbiental);
AsyncTask taskLeerIntrusos (300, true, task_LeerIntrusos);
AsyncTask taskParpadeoAlarma (300, true, task_ParpadeoAlarma);
AsyncTask taskAlarmaLedOFF (700, false, task_AlarmaLedOFF);
AsyncTask taskVentanaActivaciones(12000, false, task_VentanaActivaciones);
```

5.3 Almacenamiento en EEPROM

La contraseña de acceso y el rango horario permitido se almacenan en la EEPROM interna del Mega. Al inicio, si la EEPROM está virgen (0xFF), se cargan los valores por defecto de pinout.h. Las direcciones utilizadas son:

- Dirección 0–3: contraseña de 4 dígitos (1 byte por dígito)
- Dirección 4: hora de inicio del rango de acceso (0–23)
- Dirección 5: hora de fin del rango de acceso (0–23)
- Dirección 6–9: UID del tag RFID autorizado (4 bytes)

5.4 Autenticación multi-factor

El acceso requiere la validación simultánea de tres condiciones: contraseña correcta (comparación byte a byte con strncmp), UID RFID válido (comparación byte a byte del array uidLeida con RFID_AUTHORIZED_UID) y hora actual dentro del rango horario almacenado en EEPROM. Solo cuando las tres son verdaderas se emite SIG_CLAVE_CORRECTA.

6. Herramientas de Desarrollo

- **Arduino IDE:** Entorno de desarrollo para programar el Arduino Mega 2560. Concatena automáticamente todos los archivos .ino del mismo directorio.
- **GitHub:** Control de versiones del código fuente. Repositorio:

- **Wokwi:** Simulador de proyectos Arduino en línea, usado para validar la lógica de la MEF antes de las pruebas con hardware físico y también para el diagrama de conexiones.
- **Monitor Serial:** Herramienta de depuración integrada. Velocidad configurada a 115200 baudios.

7. Proceso de Pruebas

7.1 Backlog de pruebas (12 casos)

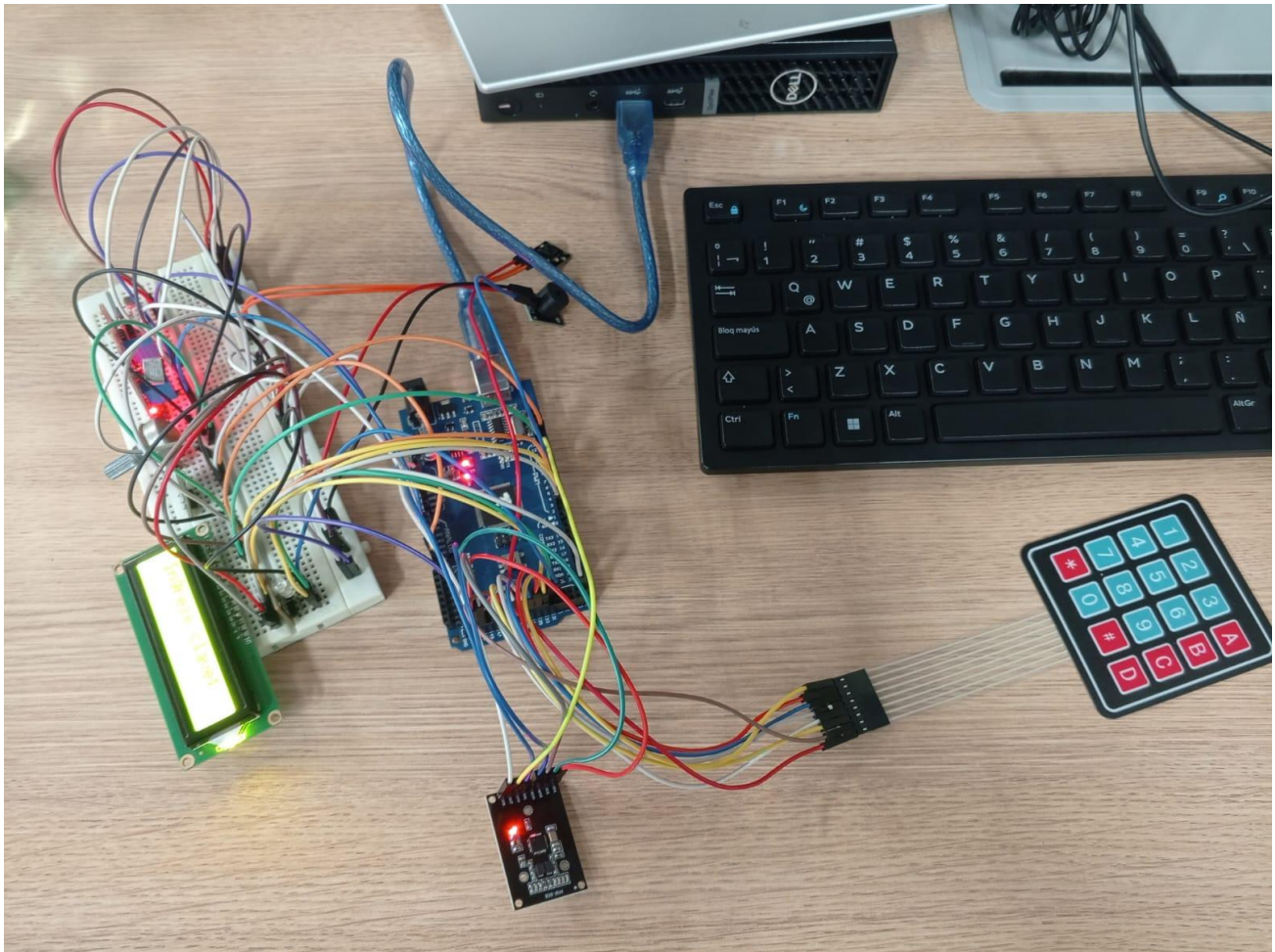
Se realizaron 12 pruebas sobre el sistema completo para verificar el comportamiento de cada transición y funcionalidad. Los resultados son los siguientes:

#	Prueba	Resultado esperado	OK	Observaciones
1	Acceso con clave correcta + RFID válido	Transición Inicio → Config	SI	Acceso concedido correctamente; LED verde brevemente activo
2	3 intentos fallidos de contraseña	Sistema pasa a estado Bloqueo	SI	LED rojo 100/500 ms; pantalla muestra SISTEMA BLOQUEADO
3	Desbloqueo con botón físico	Retorno al estado Inicio	SI	Sistema reinicia solicitud de contraseña
4	Lectura temperatura con DHT11	Valor T° visible en LCD cada 500 ms	SI	Valores coherentes con temperatura ambiente real
5	Lectura luz con fotoreistor	Valor ADC visible en LCD	SI	Respuesta inmediata a cambios de iluminación
6	Temp < 20 °C y Luz < 100 simultáneos	Transición Monitor Ambiental → Alarma	SI	Buzzer 1000 Hz activado; LED rojo parpadea 300/700 ms
7	Hall activado 3 veces seguidas	Transición Monitor Intrusos → Alarma	SI	alarmaVieneDelIntrusos = true; timeout 2 s al salir
8	Timeout 5 s en Monitor Intrusos	Retorno automático a Monitor Ambiental	SI	Temporización con AsyncTask precisa
9	Timeout 4 s en Alarma (origen ambiental)	Retorno a Monitor Ambiental	SI	Transición correcta según origen de la alarma
10	3 activaciones de Alarma en < 12 s	Retorno al estado Inicio	SI	Ventana de 12 s funcional; sistema reinicia autenticación
11	Tecla # desde Config	Transición directa a Monitor Intrusos	SI	Activación de taskLeerIntrusos y timeout 5 s
12	Tecla * desde Monitor Ambiental	Retorno a Config	SI	Transición limpia; tareas ambientales detenidas

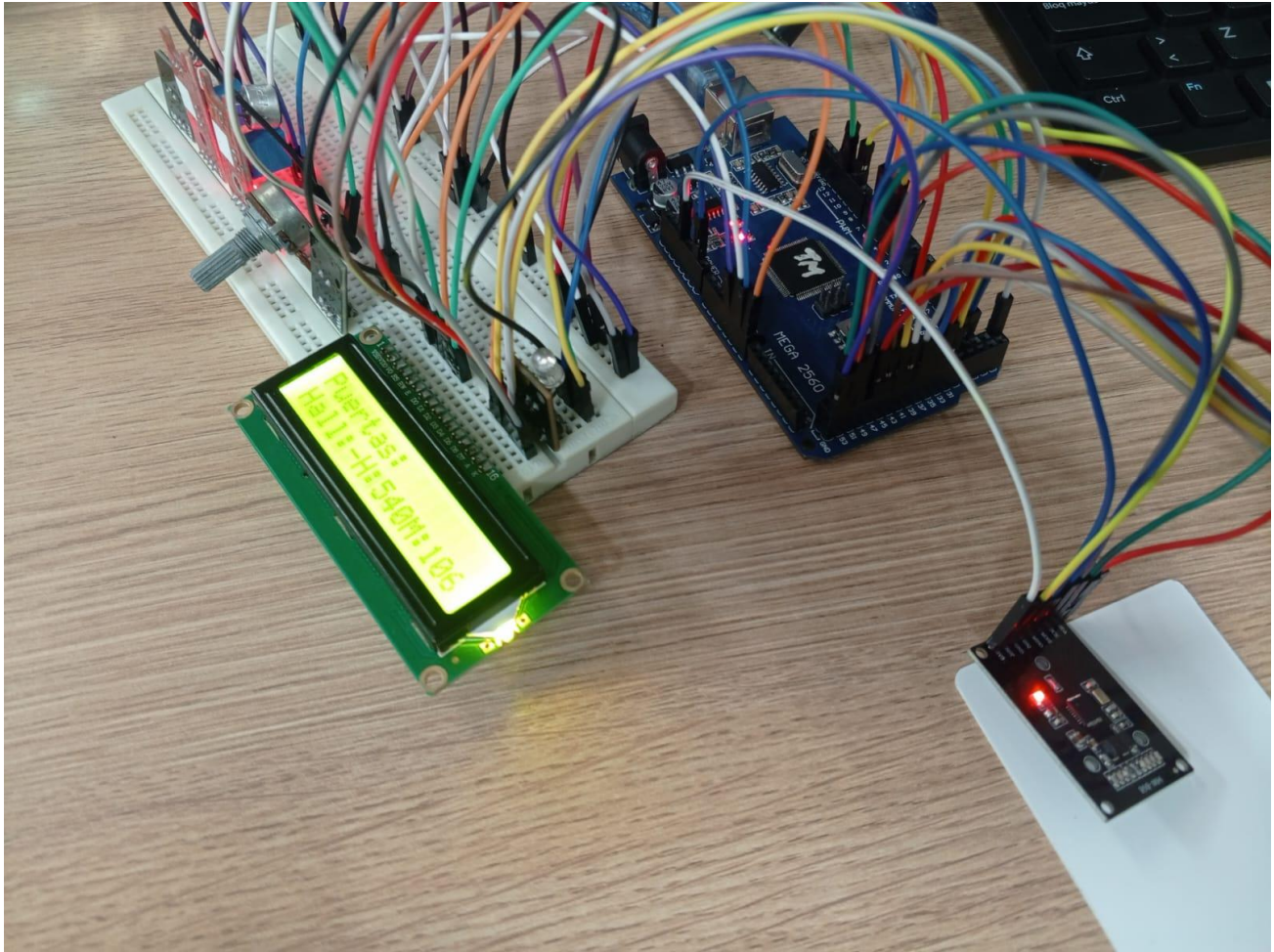
7.2 Análisis de resultados

Las 12 pruebas ejecutadas dieron resultados satisfactorios. El sistema demostró un comportamiento robusto en transiciones de estado, gestión de timeouts y alertas visuales/sonoras. La eliminación de `delay()` mediante `AsyncTaskLib` fue fundamental para la ejecución concurrente de parpadeo de LED y lectura de sensores sin interferencias.

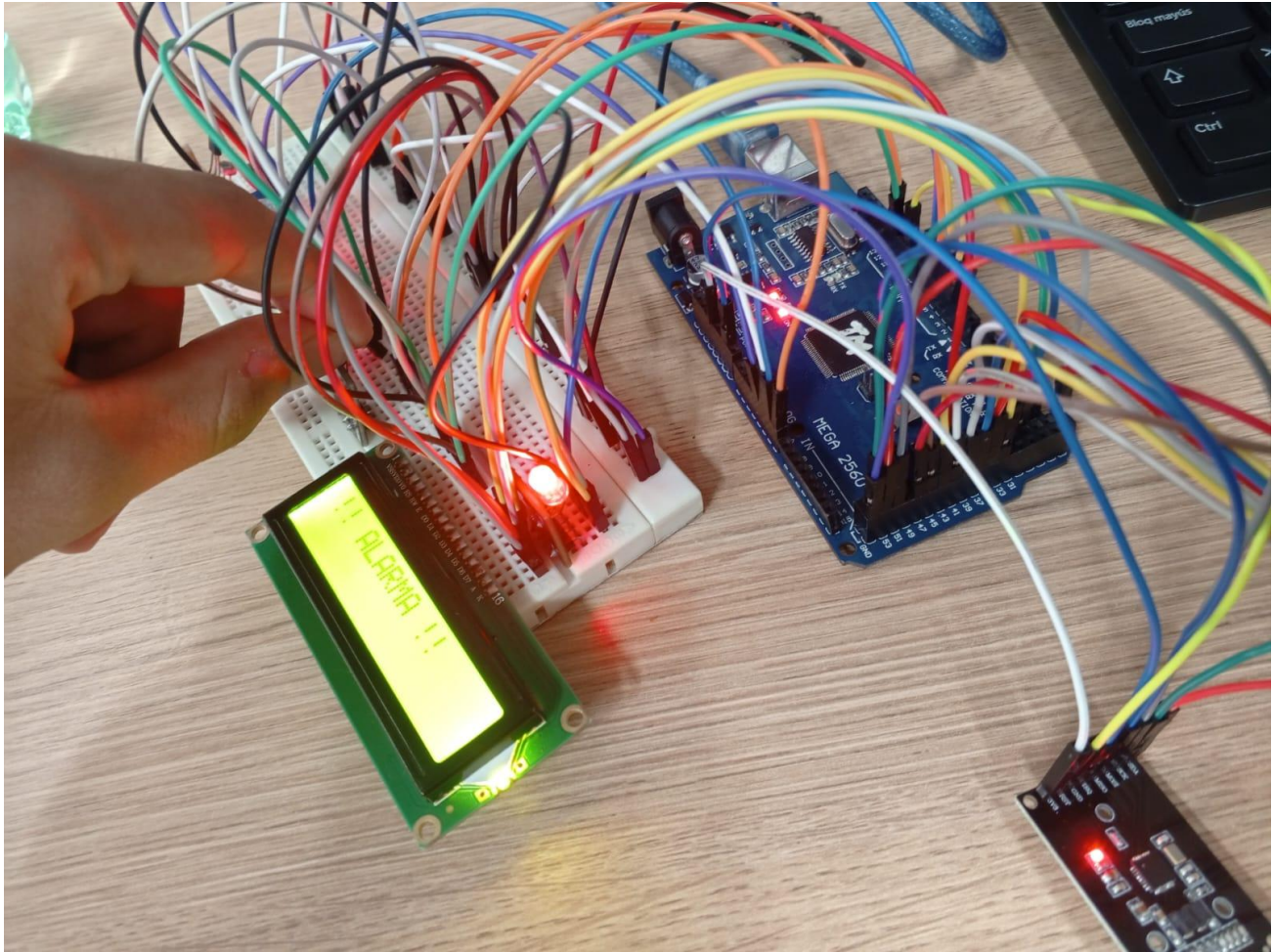
7.5 Evidencias fotográficas



Fotografía del Monitor en estado inicial, a la espera de recibir la contraseña maestra, o la tarjeta RFID.



Fotografía del Monitor en estado de lectura de los sensores.



Fotografía del Monitor en estado de alarma debido al cruce de alguno de los umbrales de los sensores.

8. Problemas Encontrados y Soluciones

8.1 Hardware

- **Puertos defectuosos en protoboard y Arduino:** Algunos pines y pistas no hacían contacto. Solución: verificación sistemática con multímetro y reubicación de componentes.
- **Componentes del laboratorio con fallas:** Módulos de sensores prestados presentaban lecturas inconsistentes. Solución: prueba de múltiples ejemplares y selección de los funcionales.
- **Conflicto del pin 11 con LCD_EN:** El pin 11 del Arduino es usado por el LCD como Enable, incompatible con tone(). Solución: LEDs y buzzer reubicados en pines 7, 9, 44 y 46.
- **BSOD en equipos al conectar Arduino:** En algunos portátiles de laboratorio, la conexión USB del Arduino causaba pantallazos azules de Windows. Solución: uso de equipos con controladores USB actualizados.

8.2 Software

- **Wokwi sin módulo RFID:** El simulador no incluye el MFRC522. Solución: la lógica RFID se validó exclusivamente en hardware físico.
- **Ventana de activaciones de alarma:** La sincronización entre el contador de activaciones y la tarea de timeout de 12 s requirió cuidado especial. Solución: taskVentanaActivaciones reinicia el contador automáticamente al expirar.
- **Lectura de humedad DHT11 en valores bajos:** El sensor DHT11 presenta errores fuera de su rango especificado (20–90 % HR). Solución: documentación de la limitación y ajuste de umbrales al rango válido del sensor.

9. Conclusiones

- La arquitectura MEF demostró ser altamente efectiva para organizar la lógica de un sistema embebido complejo, facilitando la depuración y la escalabilidad del código.
- El uso de AsyncTaskLib en lugar de delay() fue determinante para lograr una ejecución concurrente correcta, especialmente en estados con múltiples temporizaciones simultáneas.
- La autenticación multi-factor (contraseña + RFID + horario) proporciona un nivel de seguridad adecuado para aplicaciones de control de acceso embebido.
- La persistencia en EEPROM aporta robustez frente a reinicios no planeados sin requerir hardware adicional.
- Las 12 pruebas ejecutadas confirmaron la correcta funcionalidad de todas las transiciones y la precisión de los temporizadores configurados.

10. Repositorio del Proyecto

El código fuente completo está disponible en:

https://github.com/BastianRu/Proyecto_Monitor_Ambiental